

GDRM Ecosystem — Complete Technical Specification & Architectural Documentation

Project Name: GDRM Document Protection Ecosystem

Author: Gauraang Dalal

Core Framework: Flutter (Dart) & Native OS Platform Integration

Backend & Database: Firebase Authentication, Cloud Firestore & Gmail SMTP

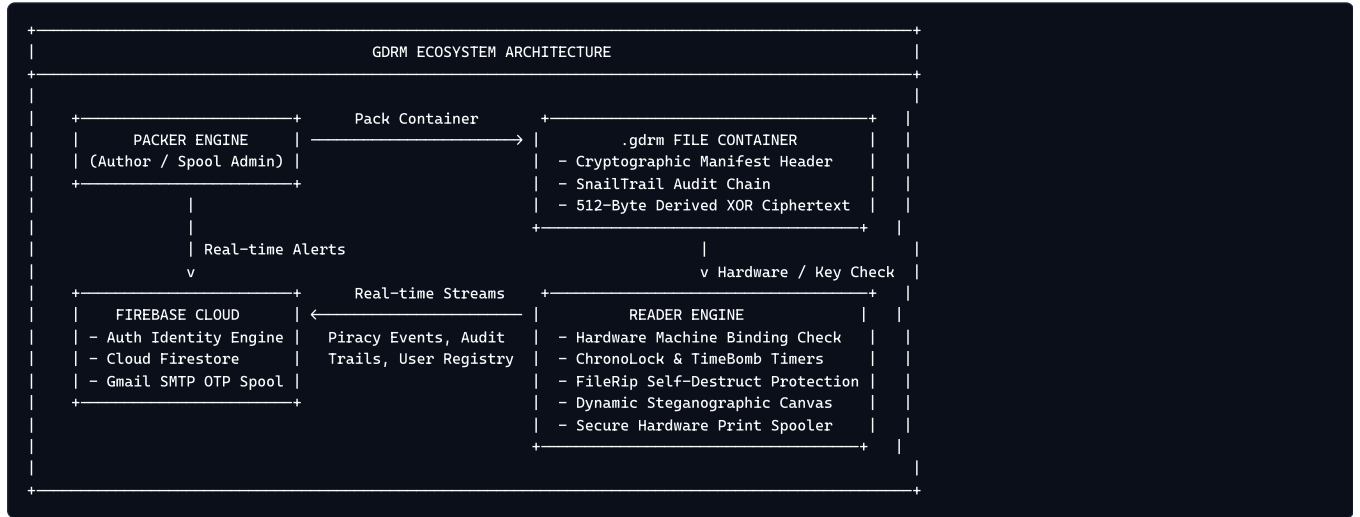
Supported Platforms: Windows (x86_64), Android, macOS, Linux, iOS

Document Classification: Comprehensive System Architecture, Protocols, Cryptographic Formalisms & Mathematical Formulas

1. Executive Summary & Architectural Overview

The **GDRM (Gaur DRM) Ecosystem** is an autonomous, cross-platform Digital Rights Management and document security architecture designed to prevent unauthorized distribution, piracy, dynamic memory extraction, screen scraping, and illicit printing of confidential PDF documents.

Unlike traditional cloud DRM systems that enforce heavy server-dependent viewer locks or proprietary browser plugins, GDRM encapsulates PDF payloads into tamper-evident, self-contained binary envelopes (`.gdrm` files). The container operates autonomously through an embedded cryptographic header, machine-bound hardware locks, dynamic steganographic watermarking, time-based cryptographic gating (ChronoLock), self-destructing brute-force barriers (FileRip), and cloud-synchronized audit trails (SnailTrail).



2. Mathematical Formalisms & Cryptographic Formulas

This section specifies all cryptographic algorithms, key derivation functions (KDF), pseudo-random generators, and mathematical transforms implemented across the GDRM Ecosystem.

2.1. Cryptographic Hash Primitives

Let $\mathcal{H}(M)$ denote the standard SHA-256 cryptographic hash function outputting a 256-bit (32-byte) digest:

$$\mathcal{H}(M) = \text{SHA-256}(M) \in 0, 1^{256}$$

Let $\text{Hex}(B)$ denote the lowercase hexadecimal encoding function mapping a byte array $B \in 0, \dots, 255^k$ to a string of length $2k$.

2.2. Machine Identity & Hardware Key Derivation

Every host device running the GDRM client is assigned a persistent, cryptographically generated UUID $U \in 0, 1^{128}$ conforming to RFC 4122 Version 4 (pseudo-random UUID).

The Machine Hardware Device Key K_{device} is derived by hashing the hardware UUID salted with the namespace constant `GDRM_DEVICE_SALT_v1:` and truncating the hexadecimal digest to the first 48 characters (192 bits of entropy):

$$K_{\text{device}} = \text{Hex} \left(\mathcal{H} \left(\text{"GDRM_DEVICE_SALT_v1:"} \parallel U \right) \right) [0 : 48]$$

$$\text{Domain}(K_{\text{device}}) \in [0 - 9a - f]^{48}$$

2.3. FileRip Password Derivation & Salt Hashing

When password protection is enabled by the author, the plaintext password P_{plain} is salted and hashed into a 64-character SHA-256 hex string H_{pwd} :

$$H_{\text{pwd}} = \text{Hex} \left(\mathcal{H}(\text{"GDRM_PASSWORD_SALT_v1:"} \parallel P_{\text{plain}}) \right)$$

The client verifies an entered attempt P_{try} by evaluating:

$$\text{VerifyPass}(P_{\text{try}}) = \begin{cases} \text{True,} & \text{if } \text{Hex} \left(\mathcal{H}(\text{"GDRM_PASSWORD_SALT_v1:"} \parallel P_{\text{try}}) \right) = H_{\text{pwd}} \\ \text{False,} & \text{otherwise} \end{cases}$$

2.4. Document Encryption: 512-Byte Keystream Derivation Pipeline

The document payload is encrypted using a cyclical pseudo-random keystream generated through continuous SHA-256 hash chaining seeded with an author-generated 128-bit random sender token T_{sender} .

1. Sender Token Generation:

$$T_{\text{sender}} = \text{Hex} (R), \quad R \xleftarrow{\$} 0, \dots, 255^{16}$$

2. Hash Chain Expansion:

To create a high-diffusion 512-byte (4096-bit) keystream K_{xor} , the system computes a sequence of SHA-256 blocks $C_0, C_1, \dots, C_{15}$ such that:

$$C_0 = \mathcal{H}(\text{"GDRM_PDF_ENCRYPT_v1:"} \parallel T_{\text{sender}}) \in 0, \dots, 255^{32}$$
$$C_k = \mathcal{H}(C_{k-1}) \in 0, \dots, 255^{32}, \quad \text{for } k = 1, 2, \dots, 15$$

3. Keystream Assembly:

The complete keystream K_{xor} is formed by concatenating the 16 consecutive 32-byte digest blocks:

$$K_{\text{xor}} = C_0 \parallel C_1 \parallel C_2 \parallel \dots \parallel C_{15} \in 0, \dots, 255^{512}$$
$$\text{Length}(K_{\text{xor}}) = 16 \times 32 = 512 \text{ bytes}$$

2.5. Symmetric XOR Cipher Stream Transformation

Given a plaintext PDF document consisting of an ordered byte array $\mathbf{P} = [P_0, P_1, \dots, P_{N-1}]$ where $P_i \in 0, \dots, 255$ and $N \in \mathbb{N}$:

1. Encryption Transformation:

$$\mathbf{E} = [E_0, E_1, \dots, E_{N-1}]$$
$$E_i = P_i \oplus K_{\text{xor}}[i \bmod 512], \quad \forall i \in 0, 1, \dots, N-1$$

where $\oplus$ represents the bitwise exclusive-OR (XOR) operator.

2. Decryption Inversion:

Due to the self-inverting algebraic property of the XOR group $(\mathbb{Z}^{2^8}, \oplus)$: $P_i = E_i \oplus K_{\text{xor}}[i \bmod 512] = (P_i \oplus K_{\text{xor}}[i \bmod 512]) \oplus K_{\text{xor}}[i \bmod 512] = P_i$

2.6. Mathematical Representation of Access Control Gates

Let $t \in \mathbb{R}^+$ represent current UTC epoch time in seconds.

1. ChronoLock Gating:

$$\text{IsTimeLocked}(t, t_{\text{openAt}}) = \begin{cases} \text{True (Locked),} & \text{if } t < t_{\text{openAt}} \\ \text{False (Unlocked),} & \text{if } t \geq t_{\text{openAt}} \end{cases}$$

2. RiggedFile TimeBomb Lifespan:

$$\text{IsTimeBombed}(t, t_{\text{expiry}}) = \begin{cases} \text{True (Trigger Destruction),} & \text{if } t > t_{\text{expiry}} \\ \text{False (Valid),} & \text{if } t \leq t_{\text{expiry}} \end{cases}$$

3. RiggedFile View-Count Ceiling:

$$\text{IsOpenCeilingExceeded}(C_{\text{curr}}, C_{\text{max}}) = \begin{cases} \text{True (Trigger Destruction),} & \text{if } C_{\text{max}} > 0 \text{ and } C_{\text{curr}} \geq C_{\text{max}} \\ \text{False (Permitted),} & \text{otherwise} \end{cases}$$

4. Hardware Machine Binding DRM State:

Let $R \in \emptyset \cup [0 - 9a - f]^{48}$ represent the **RECEIVER** header field, and K_{local} be the opening host's hardware key:

$\text{EvaluateDRM}(R, K_{\text{local}}) = \begin{cases} \text{BoundNow (First Activation),} & \text{if } R = \emptyset \\ \text{Authorised (Owner Match),} & \text{if } R = K_{\text{local}} \\ \text{PiracyDetected (Breach Alert),} & \text{if } R \neq K_{\text{local}} \end{cases}$

2.7. Forensic Canvas Matrix & Point Scaling Transform

When rendering dynamic watermarks and converting raster pages for physical paper printing:

1. Resolution DPI to PDF Points Transform:

Standard typographic PDF space utilizes 72 PostScript points per inch. Given page pixel dimensions ($W_{\text{px}}, H_{\text{px}}$) rasterized at resolution D_{dpi} :

$$W_{\text{pt}} = W_{\text{px}} \times \left(\frac{72}{D_{\text{dpi}}} \right)$$
$$H_{\text{pt}} = H_{\text{px}} \times \left(\frac{72}{D_{\text{dpi}}} \right)$$

2. Dynamic 2D Watermark Tiling Matrix:

Let (x, y) be the untransformed canvas coordinate and θ be the rotational angle ($\theta = -0.45 \text{ rad} \approx -25.78^\circ$ on screen, $\theta = -0.58 \text{ rad} \approx -33.23^\circ$ on print).

The affine rotation matrix $\mathbf{R}(\theta)$ applied to each watermark vertex is:

$$\begin{bmatrix} x' & y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x & y \end{bmatrix}$$

Watermark tiles are distributed over the periodic grid:

Missing or unrecognized delimiter for \big

where grid strides are $\Delta X = 280 \text{ pt}$ and $\Delta Y = 300 \text{ pt}$.

3. Container File Format Specification (`.gdrm`)

A `.gdrm` file is a structured binary container consisting of three contiguous segments:

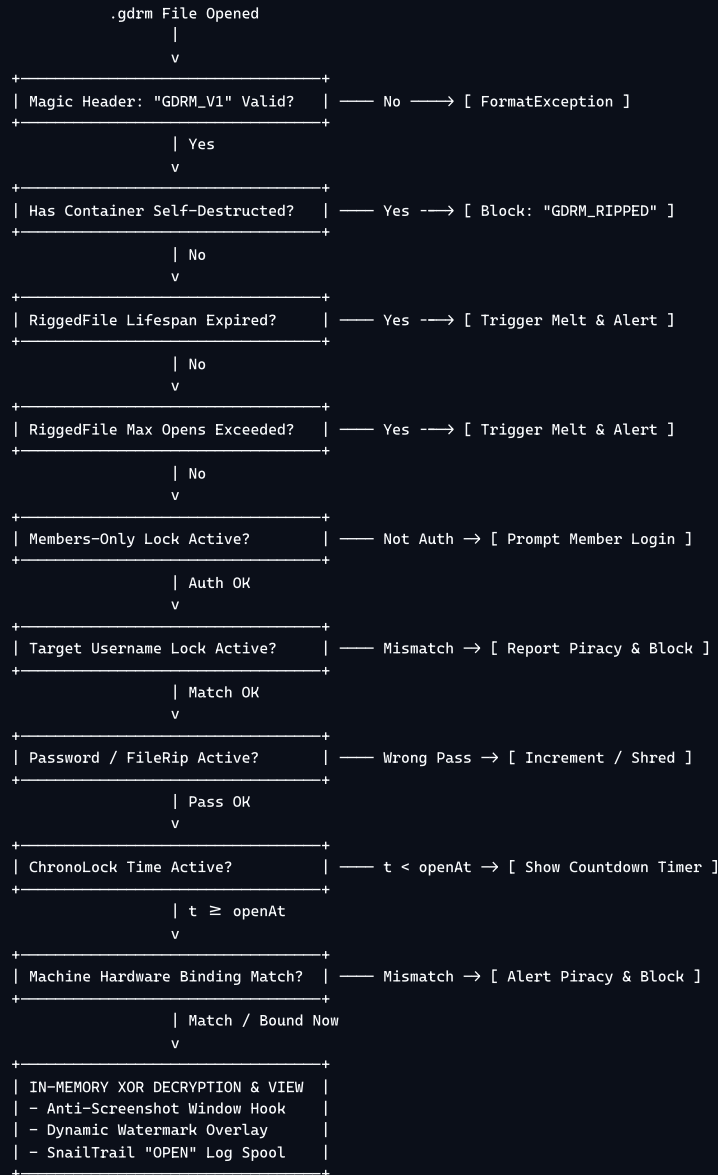
1. Plaintext Metadata Header (UTF-8 ASCII Key-Value Pairs)
2. Encapsulated SnailTrail Audit Chain Block
3. Delimiter Marker & Ciphertext Binary Payload

```
+-----+
| GDRM_V1\n
| LICENSED_TO=<Licensee Name>\n
| COPYRIGHT_OWNER=<Author Name or Organization>\n
| FINGERPRINT=GDRM-XXXX-XXXX-XXXX\n
| MEME_SIGNATURE=<Cryptographic Stamp / Meme Sig>\n
| TIMESTAMP=<Epoch Time in Seconds with 6 Decimal Precision>\n
| SENDER=<32-Hex-Char Sender Token>\n
| SENDER_USERNAME=<Firebase Author Username Handle>\n
| SENDER_EMAIL=<Author Email Address>\n
| SENDER_NAME=<Author Display Name>\n
| SENDER_DEVICE_KEY=<Origin Author Hardware Key Hash>\n
| RECEIVER=<48-Hex-Char Bound Hardware Key Hash>\n
| OPEN_AT=<ISO-8601 ChronoLock UTC Timestamp>\n
| PASSWORD_HASH=<SHA-256 Salted Password Hash>\n
| MAX_ATTEMPTS=<Integer Maximum Allowed Failed Passwords>\n
| CURRENT_ATTEMPTS=<Integer Current Failed Password Counter>\n
| TARGET_USERNAME=<Authorized Recipient Username Handle>\n
| RIGGED_EXPIRY=<ISO-8601 Lifespan Expiration UTC Timestamp>\n
| MAX_OPENS=<Integer Maximum Allowed View Count>\n
| CURRENT_OPENS=<Integer Current View Counter>\n
| ALLOW_PRINT=<1 or 0 Hardware Printing Permission>\n
| MEMBERS_ONLY=<1 or 0 GDRM Ecosystem Member Enforcement>\n
| TRAIL_START\n
| <ACTION>|<TIMESTAMP>|<DEVICE_KEY>|<DETAILS>\n
| ...
| TRAIL_END\n
| ==PDFDATA==\n
| [Binary Encrypted Payload: N Bytes of XOR-Transformed PDF Stream]
+-----+
```

Header Field Dictionary

Key	Format	Description
<code>GDRM_V1</code>	Static Magic Header	Identifies valid GDRM container version 1.
<code>LICENSED_TO</code>	UTF-8 String	Designated licensee, student, or client identity.
<code>COPYRIGHT_OWNER</code>	UTF-8 String	Author, publisher, or issuing institution.
<code>FINGERPRINT</code>	<code>GDRM-XXXX-XXXX-XXXX</code>	16-character pseudo-random cryptographic tracking hash.
<code>MEME_SIGNATURE</code>	UTF-8 String	Watermark stamp identifier or custom meme tag.
<code>TIMESTAMP</code>	Float String	Container creation UTC epoch timestamp (<code>%.6f</code>).
<code>SENDER</code>	32-Char Hex String	128-bit random seed token used to derive keystream K_{xor} .
<code>SENDER_USERNAME</code>	Lowercase Alphanumeric	Author's unique GDRM handle (e.g., <code>gaurang</code>).
<code>SENDER_EMAIL</code>	Email Address	Author's verified communication address.
<code>SENDER_NAME</code>	UTF-8 String	Author's human-readable display name.
<code>SENDER_DEVICE_KEY</code>	48-Char Hex String	Originating machine's salted hardware key.
<code>RECEIVER</code>	Empty or 48-Char Hex	Hardware key of the first activated recipient machine.
<code>OPEN_AT</code>	ISO-8601 UTC	ChronoLock unlock timestamp (empty if disabled).
<code>PASSWORD_HASH</code>	64-Char Hex String	Salted SHA-256 password hash (empty if disabled).
<code>MAX_ATTEMPTS</code>	Positive Integer	Maximum incorrect password guesses before FileRip trigger.
<code>CURRENT_ATTEMPTS</code>	Positive Integer	Number of failed password attempts recorded locally.
<code>TARGET_USERNAME</code>	Lowercase Alphanumeric	Recipient username lock (empty if unrestricted).
<code>RIGGED_EXPIRY</code>	ISO-8601 UTC	Absolute expiration datetime before automatic melt.
<code>MAX_OPENS</code>	Non-negative Integer	Maximum permitted container activations ($0 = \infty$).
<code>CURRENT_OPENS</code>	Non-negative Integer	Total successful activations recorded.
<code>ALLOW_PRINT</code>	<code>1</code> or <code>0</code>	Flag enabling/disabling physical paper printing.
<code>MEMBERS_ONLY</code>	<code>1</code> or <code>0</code>	Enforces mandatory GDRM Firebase authentication.
<code>TRAIL_START ... TRAIL_END</code>	Multi-line Block	Local SnailTrail audit entries delimited by <code> </code> .
<code>≡PDFDATA≡\n</code>	Static Delimiter	Binary boundary separating header metadata and encrypted payload.

4. Multi-Layer Security & Protection Engines



4.1. Hardware Node-Locking (Autonomous DRM Binding)

- When packed, `RECEIVER=` is initialized as empty.
- Upon first execution on an authorized device, the reader retrieves the local hardware key K_{local} via `DeviceKeyService.getDeviceKey()`.
- The container performs atomic string replacement in the binary header: `RECEIVER=\n` → `RECEIVER=<K_local>\n`.
- If the file is transferred or copied to any other computer or mobile device, the second device computes $K_{\text{pirate}} \neq K_{\text{local}}$. The reader returns `DrmStatus.piracyDetected`, instantly fires a forensic breach packet to Cloud Firestore, appends a `PIRACY_ALERT` line to the container's internal SnailTrail, and renders `PiracyScreen`.

4.2. Recipient Cloud Handle Lock (`TARGET_USERNAME`)

- Authors can bind files to specific GDRM ecosystem usernames (e.g., `@gauraang`).
- Upon opening, the reader inspects the active authenticated user's profile.
- If an unauthenticated guest or mismatched user `@pirate` attempts access, the file is rejected with full IP/MAC forensic tracking.

4.3. Members-Only Governance Lock (`MEMBERS_ONLY`)

- When enabled, access is restricted exclusively to authenticated GDRM Ecosystem community members with verified accounts.
- Anonymous and guest accounts are rejected, logging an `AUTH_REQUIRED` security event.

4.4. FileRip Anti-Brute-Force Self-Destruction

- Author specifies $M_{\max} \in [1, 10]$ attempts.
- Each failed password entry increments `CURRENT_ATTEMPTS`.
- If `CURRENT_ATTEMPTS` $\geq M_{\max}$, the reader triggers `deployedTerminalSelfDestruct()`:
 - Overwrites header with `GDRM_RIPPED_V1\NMETADATA=DESTROYED\NSTATUS=RIGGED_MELT_TRIGGERED\N...`
 - Injects 1024 bytes of cryptographically secure random noise $\mathcal{U}(0, 255)$.
 - Writes the corrupted payload back to disk, permanently destroying the original document.

4.5. ChronoLock Temporal Envelope

- Locks document visibility until an exact future UTC datetime t_{openAt} .
- Prevents premature examination paper leakage, timed press embargoes, or pre-release document access.
- Renders an interactive, real-time ticking countdown timer displaying Days, Hours, Minutes, and Seconds.

4.6. RiggedFile Lifespan TimeBomb & View Counter

- **TimeBomb:** Enforces an absolute validity window (e.g., 30 minutes, 24 hours). A background `Timer.periodic` monitors remaining duration down to 1-second intervals. If the time expires while reading, the view closes immediately, and the file on disk is overwritten with corrupted entropy bytes.
- **View Ceiling:** Enforces an absolute open count (e.g., 3 opens). Upon opening, the header counter is atomically incremented. Exceeding the threshold triggers container self-destruction.

4.7. OS-Level Anti-Screen Capture & Anti-Recording Protection

- **Windows (x86_64 / Win32):** Integrates native method channel `setSecureWindow` utilizing Windows Display Affinity APIs (`SetWindowDisplayAffinity(hwnd, WDA_MONITOR / WDA_EXCLUDEFROMCAPTURE)`). Screen capture utilities (OBS Studio, Snipping Tool, ShareX, Zoom screen share) capture a blacked-out window.
- **Android:** Native `MainActivity.kt` enforces `WindowManager.LayoutParams.FLAG_SECURE` across the activity lifecycle, disabling OS screenshots and screen recording.
- **Windows File Association:** Registers `.gdrm` extension in Windows Registry (`HKCU\Software\Classes\.gdrm` and `GDRMEcosystemFile`) to open seamlessly with the GDRM executable on double-click.

5. Forensic Dynamic Watermarking & Hardware Printing Engine

```
+-----+
|           DYNAMIC VISUAL & PHYSICAL WATERMARK LAYER           |
+-----+
| [ TOP MARGIN ]                                                 |
| [ GDRM EXAM SECURITY AUDIT - FP: GDRM-A8K2-901F-LL3P ]         |
| LICENSED: JANE DOE                                             |
|                                                                  |
| CONFIDENTIAL EVALUATION COPY - DO NOT DUPLICATE               |
| LICENSED TO: JANE DOE | RECEIVER: @JANEDOE                     |
|                                                                  |
| GDRM SECURE PHYSICAL COPY - EXAM SECURITY SYSTEM              |
| AUTHOR: PROF. SMITH | PRINT SPOOL: @JANEDOE                    |
| FP: GDRM-A8K2-901F-LL3P | IP: 192.168.1.45 | 23 Sep 2026      |
|                                                                  |
| FOR OFFICIAL USE ONLY - TRACKED CONTAINER                      |
| SIG: SIG_AUTHOR_GDRM | MAC: 3C:52:82:A1:04:90                  |
|                                                                  |
| [ BOTTOM MARGIN ]                                               |
| PRINTED BY: @JANEDOE | 23 Sep 2026 10:14 | IP: 192.168.1.45   |
| SIG: SIG_AUTHOR_GDRM                                           |
+-----+
```

5.1. Dynamic In-App Screen Watermarking (`DynamicWatermark`)

- Injected into Flutter's rendering tree via a custom `CustomPainter` layered on top of the PDF viewer widget using `Stack` and `Positioned.fill`.
- Guaranteed z -index dominance with `IgnorePointer` to preserve touch gestures, zooming, and page scrolling.
- Resolves real-time runtime hardware telemetry:
 - Client IPv4 address across all active non-loopback network interfaces.
 - Client MAC address (via `getmac` on Windows, `ip link` / `ifconfig` on Unix).
 - Operating system platform name and build.

- Active session open timestamp (`dd MMM yyyy HH:mm`).
- Licensee identity, copyright owner, and meme signature.

5.2. Secure Hardware Printing & Virtual Printer Filtering (`PrintingService`)

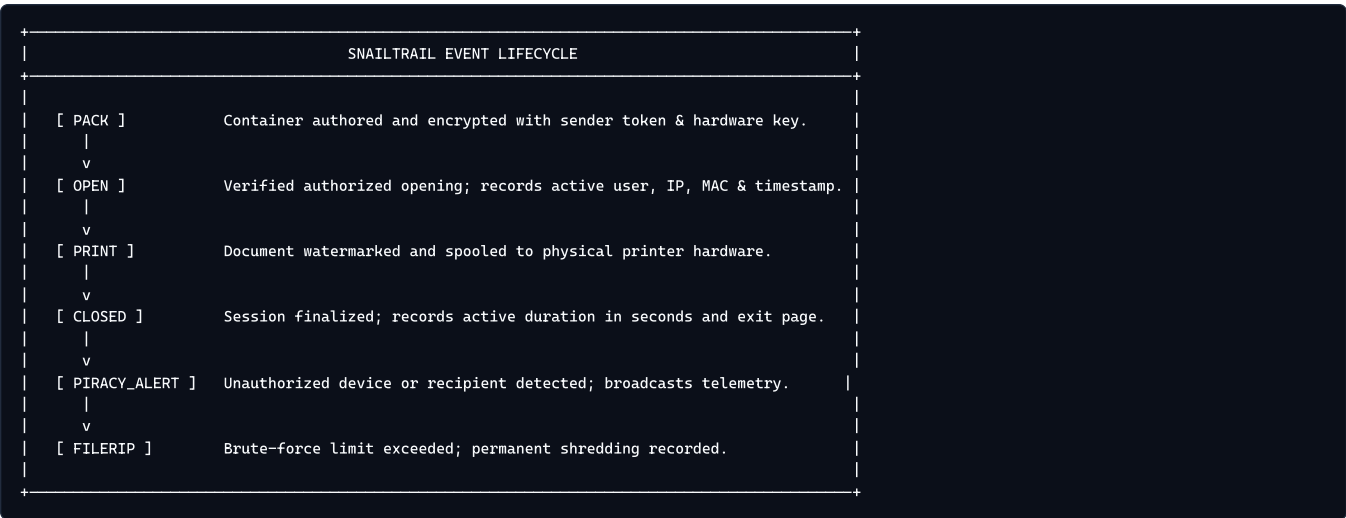
Standard PDF printing provides a significant DRM vulnerability through virtual PDF export drivers (e.g., "Microsoft Print to PDF", "Save as PDF", "CutePDF"), which output decrypted documents. GDRM implements a secure physical spooling architecture:

1. **Virtual Printer Blacklist:** Inspects device spooler queues and blocks any printer matching: `pdf`, `xps`, `onenote`, `fax`, `save as`, `print to file`, `cutepdf`, `foxit`, `bullzip`, `primopdf`, `dopdf`, `nitro`, `distiller`, `virtual`.
2. **True-to-Fit Page Rasterization:** Converts document pages into high-resolution PNG buffers (200.0 DPI) and computes exact point scaling:
$$\text{Scale} = \frac{72.0}{\text{DPI}}$$
3. **Indelible Watermark Burning:** Synthesizes a new single-spool document containing whisper-faint examination watermarks (≈ 1.5 opacity) and perimeter margins.
4. **Direct Spooling:** Sends bytes directly to physical hardware printers via `Printing.directPrintPdf` without exposing intermediary decrypted PDF files to the operating system's filesystem.

6. SnailTrail™ Audit System & Piracy Breach Telemetry

GDRM implements a dual-layer audit trail:

1. **Local Container Audit Chain:** Serialized directly inside the `.gdrm` file header.
2. **Cloud Database SnailTrail:** Live-synced to Google Cloud Firestore with real-time reactive streams.



6.1. SnailTrail Event Serialization Format

```
<ACTION>|<TIMESTAMP>|<DEVICE_KEY>|<DETAILS>
```

Example Log Entry:

```
PACKED|1727065420.104|8f3a9e041d8c72b5a190ef12bc44391a876402ec98b0f1aa|Encrypted container authored by @gauraang
OPEN|1727065512.430|8f3a9e041d8c72b5a190ef12bc44391a876402ec98b0f1aa|Session active by @janedoe (IP: 192.168.1.15, MAC: 3C:52:82:A1:04:90)
PHYSICAL_PRINT|1727065600.812|8f3a9e041d8c72b5a190ef12bc44391a876402ec98b0f1aa|Document watermarked and spooled to printer "HP LaserJet Pro MFP" by @j
CLOSED|1727065740.990|8f3a9e041d8c72b5a190ef12bc44391a876402ec98b0f1aa|Session closed. Duration: 228s, ExitPage: 4 (IP: 192.168.1.15)
```

6.2. Cloud Firestore Schema

Collection: `gdrm_users/{username}`

```
{
  "name": "Jane Doe",
  "username": "janedoe",
  "email": "janedoe@example.com",
  "uid": "firebase_auth_uid_string",
  "isEmailVerified": true,
  "emailVerifiedAt": "Timestamp",
  "createdAt": "Timestamp"
}
```

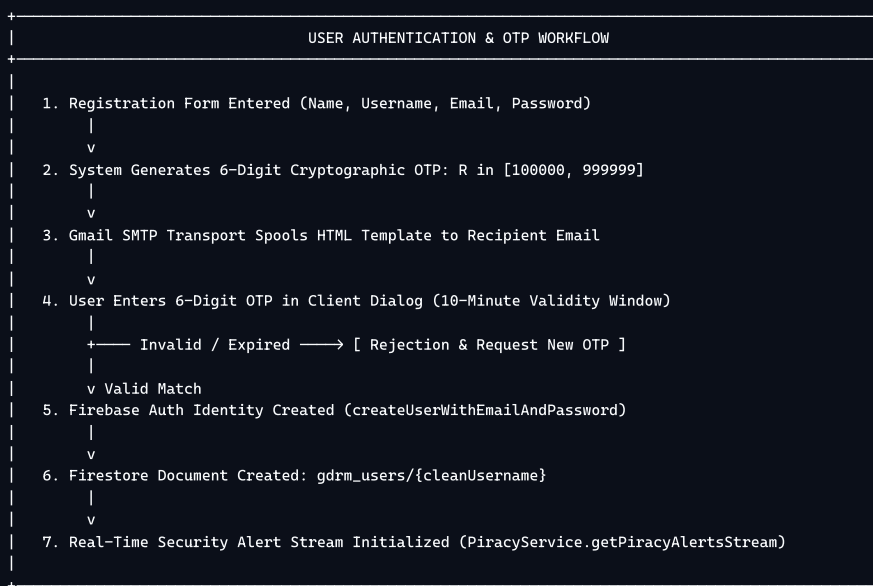
Collection: `gdrm_piracy_alerts/{alertId}`

```
{
  "fingerprint": "GDRM-K92X-P10A-88LZ",
  "fileName": "Confidential_Exam.gdrm",
  "licensedTo": "Jane Doe",
  "copyrightOwner": "University Examination Board",
  "senderUsername": "examiner_admin",
  "senderEmail": "examiner@university.edu",
  "pirateDeviceKey": "4c891f0923ba8e...",
  "pirateIp": "198.51.100.42",
  "pirateMac": "00:1A:2B:3C:4D:5E",
  "pirateUsername": "unauthorized_user",
  "piratePlatform": "windows",
  "action": "PIRACY_DETECTED",
  "details": "Hardware Piracy Breach: Device mismatch detected.",
  "timestamp": "ServerTimestamp",
  "isRead": false
}
```

Collection: `gdrm_snail_trails/{fingerprint}/events/{eventId}`

```
{
  "fingerprint": "GDRM-K92X-P10A-88LZ",
  "action": "OPEN",
  "deviceKey": "8f3a9e041d8c...",
  "details": "Session active by @janedoe (IP: 192.168.1.15)",
  "senderUsername": "examiner_admin",
  "activeUsername": "janedoe",
  "ipAddress": "192.168.1.15",
  "macAddress": "3C:52:82:A1:04:90",
  "platform": "windows",
  "timestamp": "ServerTimestamp"
}
```

7. Cloud Identity, Authentication & SMTP OTP Verification



7.1. Transactional SMTP Email Integration (`SmtplibService`)

- Connects directly to Google SMTP servers via authenticated TLS transport on port 465 / 587.
- Sends styled HTML security emails for:
 - **New Account Registration OTP**
 - **Email Address Verification OTP**
 - **Permanent Account Deletion Authorization OTP**
 - **Password Reset Recovery**

8. Codebase Directory Structure & Component Map

```
d:/Kabada/GDRM Flutter/gdrm_ecosystem/
├── android/
│   └── app/src/main/kotlin/
│       └── # Native Kotlin MainActivity (FLAG_SECURE)
├── firestore.rules
│   └── # Cloud Firestore security rules
├── lib/
│   ├── firebase_options.dart
│   │   └── # Firebase platform initialization options
│   ├── main.dart
│   │   └── # Application bootstrap & dependency injection
│   ├── models/
│   │   └── gdrm_model.dart
│   │       └── # Core data models, parsing & cryptographic transforms
│   ├── screens/
│   │   ├── chrono_lock_screen.dart
│   │   │   └── # Real-time ChronoLock countdown timer interface
│   │   ├── cloud_auth_screen.dart
│   │   │   └── # Legacy authentication screen
│   │   ├── global_auth_screen.dart
│   │   │   └── # Cloud login, registration & SMTP OTP dialogs
│   │   ├── home_screen.dart
│   │   │   └── # Responsive dual-pane / drawer layout scaffold
│   │   ├── packer_console.dart
│   │   │   └── # Authoring & packaging control console
│   │   ├── piracy_alerts_screen.dart
│   │   │   └── # Real-time author security breach monitoring hub
│   │   ├── piracy_screen.dart
│   │   │   └── # Pulsing red lock screen shown on unauthorized access
│   │   ├── reader_console.dart
│   │   │   └── # Decryption, viewing & security evaluation engine
│   │   └── snail_trail_screen.dart
│   │       └── # Audit trail visualization (Local vs Cloud stream)
│   ├── services/
│   │   ├── content_uri_service.dart
│   │   │   └── # Android content:// URI stream resolver
│   │   ├── device_key_service.dart
│   │   │   └── # Hardware UUID generation & persistent caching
│   │   ├── file_intent_service.dart
│   │   │   └── # Deep links, file drops & CLI argument stream
│   │   ├── piracy_service.dart
│   │   │   └── # Firestore audit logger & breach notification service
│   │   ├── printing_service.dart
│   │   │   └── # Virtual printer filter & watermarked physical spooler
│   │   ├── security_service.dart
│   │   │   └── # Win32 screen protection & registry file association
│   │   └── smtp_email_service.dart
│   │       └── # Gmail SMTP transport for transactional OTPs
│   ├── state/
│   │   └── app_state.dart
│   │       └── # ChangeNotifier central application state container
│   ├── theme/
│   │   └── app_theme.dart
│   │       └── # Cyan/Amber dark cyberpunk design tokens
│   ├── widgets/
│   │   ├── account_profile_dialog.dart
│   │   │   └── # Modal user profile, credentials & account deletion
│   │   ├── dynamic_watermark.dart
│   │   │   └── # Tiled affine rotation canvas watermark painter
│   │   ├── shared_widgets.dart
│   │   │   └── # Common UI components, nav buttons & auth badges
│   │   └── sidebar.dart
│   │       └── # Desktop navigation sidebar & active manifest drawer
├── pubspec.yaml
└── README.md
    └── # Quickstart documentation
```

9. Security Threat Analysis & Mitigation Matrix

Attack Vector	Threat Scenario	GDRM Mitigation Mechanism
File Sharing / Piracy	User sends <code>.gdrm</code> file to a friend or unauthorized device.	Hardware Node-Locking: Receiver hash R mismatch triggers <code>DrmStatus.piracyDetected</code> , locks display, and alerts author via Firestore.
Virtual PDF Re-export	User attempts to "Print to PDF" to strip DRM encryption.	Virtual Printer Filter: <code>PrintingService</code> scans spooler queues and blocks virtual PDF/XPS writers; only physical printers are permitted.
Screen Scraping / Recording	Attacker records screen with OBS, Snipping Tool, or Zoom.	OS Anti-Capture Hook: Windows <code>SetWindowDisplayAffinity</code> and Android <code>FLAG_SECURE</code> black out application window buffers.
Camera / External Capture	Attacker takes a physical photograph of the monitor screen.	Dynamic Forensic Watermark: Displays viewer's IP, MAC address, timestamp, licensee name, and signature across the entire document.
Physical Document Leakage	Printed paper copies are leaked outside authorized rooms.	Perimeter & Diagonal Exam Watermark: Indelible low-opacity author, printer spool user, IP, MAC, and fingerprint burned into physical paper.
Brute-Force Password Cracking	Attacker uses dictionary attacks against password-protected file.	FileRip Self-Destruction: Container records failed attempts; reaching $M_{\max}$ shreds the file on disk into 1024 random bytes.
Premature Leakage	Exam papers or embargoed files opened before scheduled date.	ChronoLock Engine: Time-locked until t_{openAt} ; evaluates UTC time and displays live countdown timer.
Container Tampering	Attacker edits header strings or attempts XOR payload extraction.	Format & Hash Validation: Tampering corrupts marker boundaries or keystream derivation, causing decryption failure.

10. Summary & Verification

The GDRM Ecosystem combines cryptographic rigor with client-side operating system protections and cloud-synchronized audit capabilities. All formulas, structural packet offsets, keystream derivation hash chains, state machines, and threat mitigations are implemented natively within Dart/Flutter and verified for cross-platform deployment.